SEDAS

Social Engineering Defence and Awareness System:

An Integrated Machine Learning Platform for Phishing Detection and User Awareness

MIS5320 Applied IS Project — Part B · Assessment 3: Final Project Report

Team	Ashraful Islam (240135); Huzaifa Iqbal (240220); Sonu (250001); Dominic Kipchirchir (240401)
Supervisor	Dr Chandana Withana
Institution	Apex Australia Higher Education
Base paper [1]	Al-Subaiey et al. (2024), Computers and Electrical Engineering, 120:109625 (Q1)
Repository	github.com/ailinkon/MIS5320-SEDAS-CyberProjectB
System version	0.1.0

Abstract

But phishing attacks are one of the most nagging social engineering risks because they are not directed at a technical control which can easily be patched or filtered. Higher education environments with large and continually changing user populations, decentralised services and access to financial and research data, to name a few, provide a larger attack surface. Detection is not enough – a classifier which can't explain its decision, can't alert unsafe links for review and fails to feed results back into training and governance simply doesn't provide organisations with full protection. During this project, you will acquire knowledge of SEDAS – a full-fledged defence and awareness portal – that includes phishing-email classification and explainable output, tamper evident verification of an email's URL, adaptive awareness training, and record keeping for governance.

The study started with an independent replication of a published benchmark on the same corpus used in the study containing 82,486 emails, and expanded to include six classification methods and four optimisation strategies. LinearSVC achieved 99.16% F1 score while other classifiers like Random Forest, LR, XGBoost and Multinomial Naive Bayes achieved 99.0, 97.47–98.71% F1 score on the paper. Five-fold cross validation was also done which had stable folds, and a paired significance test was done that yielded that the results of LinearSVC are highly unlikely to have happened by chance from its closest competitor ($t = 16.60$ and $p = 0.0001$). A technical feasibility / data efficiency study (DistilBERT) scored 98.00% F1, but not whether transformers can ever surpass the classical upper bound on the large corpus, among a much smaller subsample under a CPU-constrained setting. There were four other methods to surpass the LinearSVC benchmark (two models based on engineered features, two ensemble methods and hyperparameter tuning), however, which offered relatively modest and not necessarily significant improvements, with only the worst of this list of models showing an improvement that was significant using cross validation. This is indicating that it isn't a failure but it's the maximum that was achieved for a configuration being tested. Moreover, SEDAS also reports false-positive error, absolute number of error, confusion matrices, and observed latency, providing a different insight into the operation of the various SEDAS. All these boil down to a not to be belittled accuracy gain, a benchmarking standard that is reproducible, an empirically verified ceiling for performance and a very comprehensive system centred on validated detection, URL integrity controls, adaptive humanlayer defence and governance.

Keywords: phishing detection, social engineering, machine learning, TF-IDF, security awareness training, explainable AI, tamper-evident logging

Abbreviations

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
BERT	Bidirectional Encoder Representations from Transformers
CI	Continuous Integration
CORS	Cross-Origin Resource Sharing
CPU	Central Processing Unit
EWMA	Exponentially Weighted Moving Average
FN	False Negative
FP	False Positive
FR	Functional Requirement
GPU	Graphics Processing Unit
HTML	HyperText Markup Language
HTTP / HTTPS	HyperText Transfer Protocol (Secure)
IP	Internet Protocol
KPI	Key Performance Indicator
LIME	Local Interpretable Model-agnostic Explanations
LMS	Learning Management System
MNB	Multinomial Naive Bayes
MVP	Minimum Viable Product
NFR	Non-Functional Requirement
OWASP	Open Web Application Security Project
REST	Representational State Transfer
RF	Random Forest
SEDAS	Social Engineering Defence and Awareness System
SHA	Secure Hash Algorithm
SHAP	SHapley Additive exPlanations
SIEM	Security Information and Event Management
SMS	Short Message Service
SQL	Structured Query Language
SSO	Single Sign-On
SSRF	Server-Side Request Forgery
SVC / SVM	Support Vector Classifier / Support Vector Machine
TF-IDF	Term Frequency–Inverse Document Frequency
TLS	Transport Layer Security
TN	True Negative
TP	True Positive
URL	Uniform Resource Locator

1. Introduction

1.1 Background and Motivation

Despite their estrangement from the “settling in” factors, it is possible for one of them to become a victim to phishing even after the realization of what are best technical control measures. Especially in the university domain, which are big, constantly evolving, and provide a lot of different services which the attacker can get a lot of easily spoofable identities. Detection technology has come a long way in recent years [1] – [3] but systematic interventions are adaptable over time, versus one-off strategies (simple exposure) and they have been proven many times to be more effective than exposure strategies [4] – [5] – as well, and the effectiveness depends in large part on people, how they are trained and when. At the same time, content created by AI makes phishing messages harder to detect as it eradicates the obvious spelling and grammar errors traditionally used to spot phishing messages and makes awareness and avoidance more critical [6][7]. The principle of SEDAS is that classification should be a protective process that takes place along with URL verification, user friendly feedback, adaptive learning and audit documentation and governance.

1.2 Problem Statement

The two problems to be resolved in this project are related to each other. A first problem on the field level: Most of the experiments on phishing detection focus on data sets or setups that are not completely accessible to other researchers, and this makes it impossible for them to verify whether the results of the experiments are due to a real improvement of the detection method or simply to different data preparation. The response to this is the base study [1] which provides company write-offs on a big general public e-mail corpus. SEDAS starts with an independent reproduction with the exact same 82,486e-mail dataset alongside with fixed procedure and seed. Second, detection research in most cases focuses on a single classifier, and often "labelled, reported, done" – social engineering goes beyond the message content to judgements and actions by users and organisations. SEDAS overcomes this through explainable classification combined with tamper evident URL verification methods, adaptive training and auditable records, paving the way for the research question to be no longer about the extent to which phishing can be classified, but also about the validation of detection that can be integrated within a defensive workflow that is reproducible and operationally measurable.

1.3 Contributions

SEDAS' five contributions different from copying a published score. First an independent re-run of the [1] benchmark on exactly the same dataset showing the split, indicating the composition of the corpus, indicating parameter settings and revealing the metrics used. Second, six methods are evaluated together in a single framework: LinearSVC and Random Forest and Multinomial Naive Bayes are the classical methods that they are compared against; Logistic Regression and XGBoost are included; and DistilBERT is a radically different representation, but one which they are not yet capable of supporting, and they have separated to ensure that it is not violated of equivalence. Third, four systematic optimisation strategies were then tried following replication: engineered features, equal-weight voting, SVC-weighted voting, and regularisation tuning; all of which showed no notable increase in performance, thereby providing an empirical performance-ceiling finding. Fourth, SEDAS reports operational metrics [1] is unable to report: false-positive rates, absolute counts of errors, confusion matrices and measured latency. Fifth, the validated classifier is a part of an integrated architecture of explainable detection, tamper evident url validation and adaptive training and governance record keeping as demonstrated both empirically as well as architecturally.

2. Literature Review

2.1 Phishing and Social Engineering: Attack Landscape

Phishing has gone beyond a spoofer's ability to easily penetrate bulk spam or their marketing net into social-engineering attacks that use identity, context and urgency (spear-phishing, business-email-compromise, credential harvesting and lookalike websites), which are effective because they exploit routine organisational behaviour. In a higher education context, the attackers have successfully impersonated lecturers, administrators or platforms, and it is the same language of legitimate operations that is the “attack surface”. This is changing with the advent of generative AI, however, as Opara et al. [6] and reference [7] discuss phishing that is grammatically correct and fluent. This means that the attack volume and variation is enhanced, leading SEDAS to believe that no detector should be expected to solve the entire social-engineering attack path.

2.2 Machine Learning Approaches to Phishing Detection

Traditional machine learning model is still a good thumb rule for detecting phishing. Our baseline for this project, Al-Subaiey et al. (2016) [1] has reported 0.99 F1 using SVM-TFIDF; this outperformed the SVM using the same features and a much greater ensemble benchmark [2] where the deep learning models performed best but classical baselines such as SVM-TFIDF were competitive. This holds true of other independent studies: another evaluation of a similar dataset was built using SVM, Logistic Regression, Random Forest and Gradient Boosting, and once more, it was SVM which was found to perform best, being most suitable for high dimensional sparce vectors for emails. SEDAS methodology of staged approach is supported by this literature as they mirror the most powerful classical benchmark; extensions are then tested without altering the matrix of evaluation.

2.3 Detection based on Deep learning and Transformer-based detection

In recent years, it's been common practice to apply transformer models for phishing detection since they are able to model word context and order, which simply can't be captured in TF-IDF's bag of words approach; relevant where the same vocabulary may be benign or malicious depending on how requests, urgency, and identity claims combine. A paper directly comparing BERT and RoBERTa showed that transformers at 99.43% accuracy outperformed BERT [3] while the same family of corpora (Nazario and Enron) was used in a BERT vs RoBERTa study that found that RoBERTa produced fewer false negatives [9]. The large scale benchmark [2] though, reported an average margin of 4.7% for transformers, which shows that while it's indeed a positive, the impact is not so strong as the numbers seem to suggest. Hybrid architectures in a deep approach have also been investigated [10]. Section 5.3 is motivated by these findings, in order to compare the transformers.

2.4 Security Application with Explainable AI

Explainable AI[3, 4] overcomes the lack of trust or transparency inherent with black-box classifiers. As of LIME, being used on individual predictions as in [1] or extended across the literature: a framework which integrates XGBoost and SHAP matches a 97.78% accuracy with explainability [8]; a hybrid of LIME and SHAP using PDP has been used for the time-aware feature-extraction on ~500K web pages [11]; and explainability has been applied to IoT/cloud phishing contexts [13]. A bigger taxonomical view of explainability places it within the context of trustworthy automation instead of something merely cosmetic [14] – albeit transformer-specific explainability for SMS spam [15] and phishing

detection using LLMs [16] have been demonstrated. SEDAS follows this pattern but picks a smaller subtler mechanism which it has most done what it had the most stable linear model to do, that is, multiply the present TF-IDF feature by the trained coefficient and bring the top 10 to the surface, upholding [1]s goal of inspectable decisions, while also limiting complexity of implementation.

2.5 Security Awareness Training and Human Factors

Only technical detection devices can't solve the human vulnerabilities that social engineering takes advantage of. One study that has reviewed more than 30 studies determined that simulated training has a positive and significant effect when it is continuous with risk-adaptive lesson difficulty and reinforced, a direct benefit towards the persona-centric, risk-adaptive design of SEDAS, which automatically selects difficulty levels for lessons based on a dynamically maintained risk score instead of providing everyone the same training materials. Another complementary study noted that merely the fact of being exposed eases the transfer of behavior, as behaving with engagement or feedback has a measurable impact on behavior persistence (4). Repetitive exercises lead to fatigue. This drives SEDAS's immediate feedback on a question-by-question basis, so when someone gets one of the questions wrong, they learn from it, and persona-based feedback containing information about the risks faced by pupils, teachers and administrators, consistent with the theory of planned behaviour basis used in the Part A design.

2.6 Trust and Integrity Verification Mechanisms

Records of URL's classification provided with blockchain and hash-chaining techniques as tamper evident. At the same time, MLPhishChain has been devised, which is an integration between ML classification and blockchain technology (Ethereum in particular), with smart contracts used to immutably store the classification of URLs: directly paralleling SEDAS's URL verification service [17]. Instead of being IoT dependent, like MLPhishChain, which needs Ethereum smart contracts and wallet interactions for updates, SEDAS's refinement register (R-01) is a registry based on the SHA-256 hash where each entry is mandatory, and if the one that created it were to change, the change would be cryptographically detectable, neither wallet-based nor IoT-dependent, and without the need for any gas fees or wallet infrastructure, or latency from blockchain transactions. This isn't really a blockchain as there's no decentralised consensus, but they've effectively warned that there's no decentralisation, and that if somebody wanted to change the data and recompute the chain then they could. The refinement was selected due to the lack of the need to add engineering overhead to first support distributed-ledger infrastructure, which would not be assessable in the MVP behaviour, until future production work was completed where distributed consensus was needed.

2.7 Research Gap

Based on the literature and the base paper [1] three gaps led to the motivation for this study. Reproducibility: a lot of surveyed literature doesn't make all preprocessing and configurations transparent; SEDAS promises a complete reproducibility using the same corpus, split and set of seeds and the same evaluation protocol as in [1]. Secondly integration: all studies reviewed address detection, explainability, training or integrity verification with only one of the studies addressing all of these components in isolation in the evaluated system, which is SEDAS's particular contribution. Third, there is not yet consensus on whether or not the classical approach relates better - [2] shows transformers are more effective by much, [9] shows that transformers are more effective by a little,

and [3] shows that transformers are no more effective. When combined these gaps change the focus from a slightly better accuracy towards identifying what is repeatable, what additional optimisation can be achieved, and how detection can contribute to overall defence.

3. Methodology

These are the steps that were followed in the Base paper: research design, architecture, datasource, preprocessing, algorithms, extension experiments, transformer approach, evaluation protocol and refinements done to the original design.

3.1 Research Design

The study had a two stage design using independent replication followed by systematic extension because an improvement claim is only meaningful if an improvement claim in the pipeline can be demonstrated to replicate the published benchmark. SEDAS reused the same public corpus as [1]; it rerun the TF-IDF pipeline for SE for all three of its methods and added alternatives afterwards. The development lifecycle was modelled after the Unified Process (Inception/Elaboration for specification review, Construction for implementation, Transition for testing/evolution) and the Agile practices (short sprints, feature branches, continuous testing) were done within the context of the process stages. The same cleaned corpus was used for all classical and extension experiments and the stratified 80/20 split (and common evaluation measures) was the same for each experiment, except for DistilBERT, which employed a CPU-tagged subsample, as the results of which are detailed separately.

3.2 System Architecture

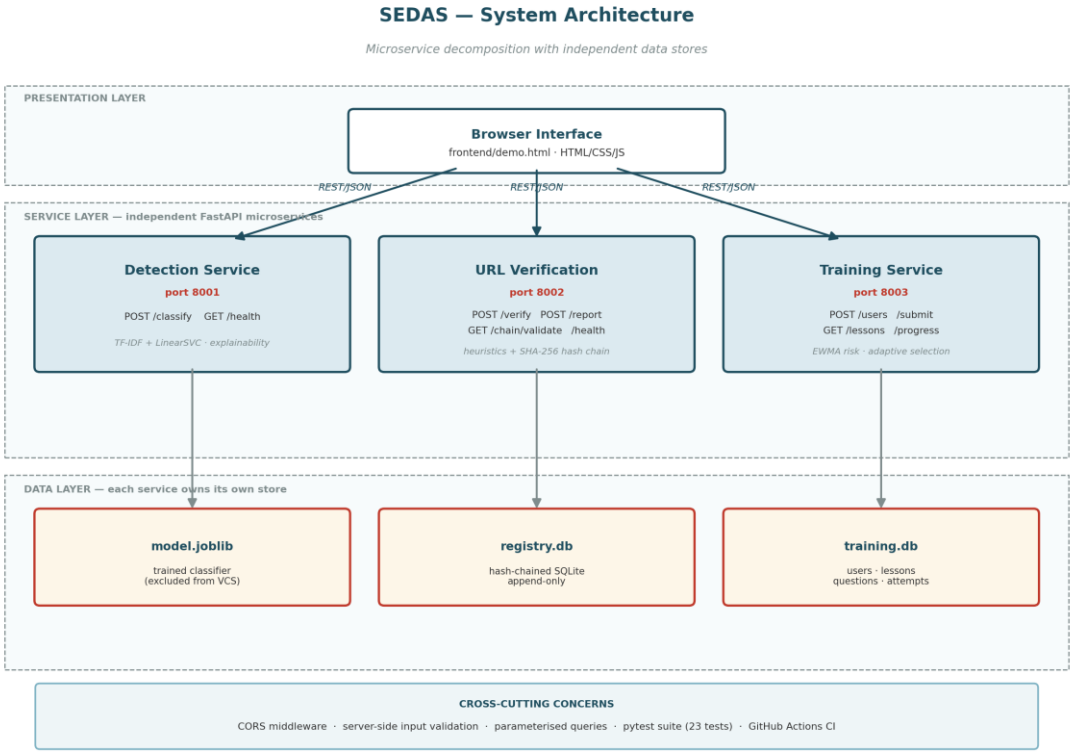

Figure 1: SEDAS system architecture — three-layer decomposition with independent per-service datastores.

The architecture of SEDAS is a three layer approach, with presentation being the Browser user interface without any logic for decision making; service being three separate FastAPI apps – one for stateless detection; one for a "write heavy" tamper evident URL verification; and one for stateful adaptive training; and data being independent per service, with a serialised model artefact for detection, independent SQLite container for URL; and for training. Keeping the microservice decomposition was settled on due to different runtimes - detection needs lots of compute and is write-poor and stateless; URL verification has an integrity constraint and is write-heavy; training has a per-user state and is read-heavy, decomposing separates them for independent scaling, testability and failure isolation as MVP cost; three processes instead of one, no gateway or service discovery. The different elements of communication are HTTP/JSON for all their operations, and FastAPI automatically generates OpenAPI specs based upon the endpoint as well as the validation model.

3.3 Dataset

Table 1: Dataset composition.

Attribute	Value	Note
Total emails	82,486	after removal of null entries
Phishing	42,891	matches base paper exactly

Attribute	Value	Note
Legitimate	39,595	matches base paper exactly
Source corpora	Six merged	Enron, Ling-Spam, CEAS 2008, Nazario, Nigerian Fraud, SpamAssassin
Fields used	text_combined, label	subject and body merged; binary label
Availability	Public	freely available, supporting reproducibility

The dataset has been collected from 6 public sources: Enron, Ling-Spam, CEAS 2008, Nazario, Nigerian Fraud and SpamAssassin, but since email collections are prone to null emails, this has left total of 82,486 usable emails (42,891 phishing and 39,595 legitimate). The only important reason to go for this corpus is that it is the one that has been published with [1] – the quantity of classes loaded was exactly the same as that reported in the benchmark, so that if there are differences between the results we obtain from this corpus and the benchmark, they have to be because of the way the pipeline is set up, and not the corpus itself. The public availability also aids with the problem of transparency that it identifies - the ability to reproduce the data. In exchange, external validity should be considered since these are research collections, and may not be representative of how language and distribution of attacks are used in actual organisations — the domain-shift is explored in more detail in Section 5.4.

3.4 Data Model

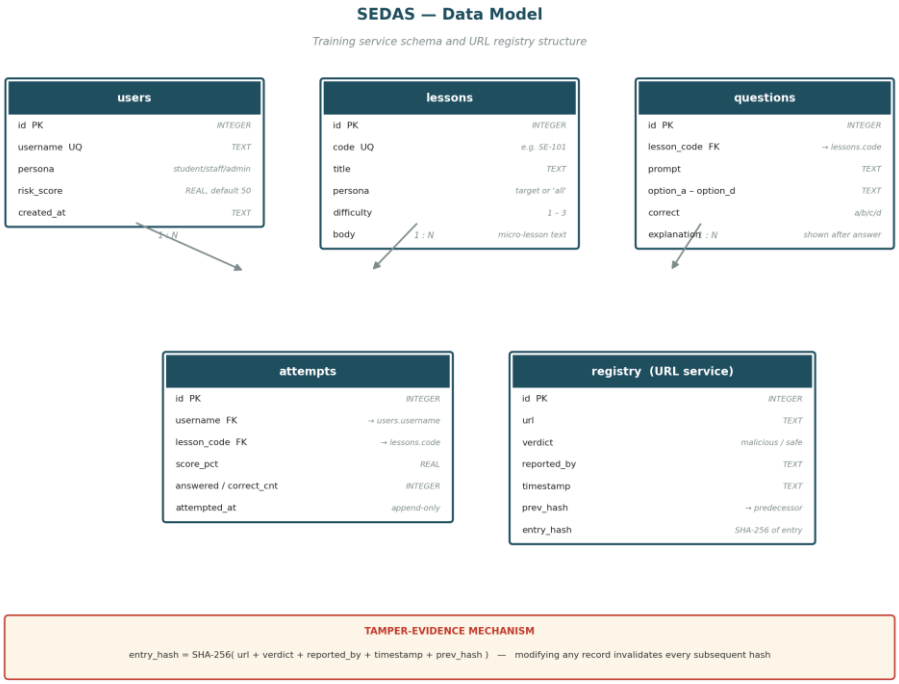

Figure 2: Entity structure for the training service and the URL verification registry.

The hash is calculated for each registry entry replacing content (URL, verdict, reporter, timestamp) with the previous entry's so any change to the content of a registry entry causes the hash of that entry and all the subsequent ones until the end to become invalid. The training service comprises 4 relational tables, namely, users (username, persona, risk score), lessons (code, persona, difficulty, content), questions (links to a lesson, the quiz interface returns only the quiz question text and response options, the correct quiz answer is kept on the server for later analysis and the audit of attempts), and attempts (append only, logging every completed quiz for audit and trend reconstruction). The URL verification service is actually a separator between the URL and the verdict and will have a separate registry table that goes as follows - (URL, verdict, reporter, timestamp, previous hash, entry hash) with the first table having GENESIS as its predecessor and the validation endpoint will walk the chain and recompute each hash thereby detecting modifications. This offer Tamper Proofness not Decentralised Consensus – theoretically it could be possible for a sufficiently authorised party to have access to the database and modify the records and rework the chain at MVP, which has been accepted as a limitation in exchange for not having to deal with blockchain infrastructure overheads.

3.5 Preprocessing and Feature Extraction

For the email text, TF-IDF was used with a max features of 50,000, an n-gram range of 1-2 and with the removal of English stop-words; the importance of bigrams for phishing intent detection is that these intentions are often communicated in succinct phrases like "urgent action" or "verify account". The resultant matrix is relatively high dimensional, contains the sparse data from only a few positions in each of the emails' 50000 entries and, most significantly, this happens after each of the datasets has been processed just a single time, while the XGBoost model's weaker performance in Section 5.2 is partly because each of the datasets has only a few partitions to learn

in the mostly-zero feature space. A separate extension experiment was run to determine whether adding information in the form of hand-crafted signals to the TF-IDF representation aids decision making: The signals used were TF-IDF-normalised urls (URLs made common on a token basis) stripped of HTML, and were six numeric features representing the length of the message, the number of urls, exclamation mark count, digit count, capital letter ratio, and the number of times phishing words appeared in the message.

3.6 Algorithms Investigated

Five classical algorithms were tested with the same and common TF-IDF pipeline. LinearSVC, Random Forest and Multinomial Naive Bayes are the identical to the families of models used in the benchmark study [1]. Due to the published result, Logistic Regression and XGBoost were added as extensions in order to see if there is a better linear method (Logistic Regression) or a sequentially boosted tree ensemble (XGBoost) that will outperform the published result. Thus, each algorithm is used for comparison for its different usage: Maximum margin linear separation, bagged nonlinear trees, probabilistic word evidence, linear probability estimation and gradient boosting. The last method is to substitute the sparse TF-IDF features with the contextual representations of the tokens in the document, as done by DistilBERT (Section 3.8).

3.6.1 Support Vector Classifier (LinearSVC)

LinearSVC is a support vector classifier that decides on the boundaries by using a linear boundary. is a learning process that finds a hyperplane that separates out the phishing and legitimate examples and maximises the distance to the closest support vectors in order to make it more robust. The method works quite well for TF-IDF as it can accommodate the high dimensional sparse vectors typically generated by text classifiers and can indulge into learning weights for many informative terms without the use of an expensive nonlinear kernel. The method that was most used to replicate the benchmark was linearSVC due to its being the best reported by SVM classifier in [1].

3.6.2 Random Forest

Random Forest is a combination of decision trees that were trained using bootstrap samples. Majority voting: Each tree makes a class prediction and the forest makes a class prediction based upon majority voting. Random feature selection and bootstrap sampling helps to decrease the problem of overfitting, while enabling nonlinear interactions. The 100 estimators were utilized with random_state=42 which was done in the benchmark. Random Forest was added as another replicate (immediate) method and compared to LinearSVC.

3.6.3 Multinomial Naive Bayes

A popular probabilistic classifier over word-frequency and TF-IDF features is the multinomial Naive Bayes that calculates the probability in a specific feature in phishing and legitimate classes. I guess its efficiency depends on an assumption of conditional independence: In order to be efficient, the class has to be known and every word within a class has to be independent in terms of representing any relationships between words or the word order. The choice to use MNB was due to its evaluation in [1] and consequences of this assumption are explored in Section 5.1.

3.6.4 Logistic Regression (extension)

Logistic Regression is a linear probabilistic classifier that simply takes the vector of TF-IDF features (the features computed), applies some weights to them and passes the result through the logistic function to calculate the probability of phishing. It is efficient on sparse data, and is interpretable, when compared to the LinearSVC. Because it was not evaluated in [1] it was added as an extension

to compare the performance of a linear model based on the probability concept with SVC with the same Corpus and Split.

3.6.5 XGBoost (extension)

XGBoost is an implementation of gradient boosted decision trees: taking each successive tree to reduce the error from the previous set of trees as the ensemble. This can be used to model nonlinear interactions which cannot be modeled by a linear classifier. SEDAS was able to use 200 estimators, and its maximum depth was 6 and the learning rate is 0.1. As it was not evaluated in [1] it was included to check a method which was not used in the benchmark as well as to check if boosted-tree partitioning is suitable to the extremely sparse 50 000 dimensional representation used here.

3.7 Extension Experiments

Finally, four controlled experiments were conducted that varied plausible methods of surpassing the SVC baseline, but maintained the same dataset, splitting and evaluation process to show that this is a reproduction of the baseline and no real contribution is found in reproduction alone. Experiment 1 aggregated TF-IDF with six created security signals (number of URLs, ratio of capital letters, number of phishing-keywords, and more), with the assumption that explicit signals might be able to convey information which is not gained by lexical TF-IDF. In experiment 2, an ensemble of all three classifiers (SVC, Random Forest and Logistic Regression) where each contribution was equal in weight was constructed to overcome the limitations of individual classifiers, with an expectation that different classifiers will make decisions independently, to compensate for each other. Experiment 3 also gave the same ensemble 3:1:1 (SVM:SVC) weightage of SVC's opinion over any other sources that decide a case, assuming that SVC must be dominating if a case gets decided between ambiguous cases, otherwise being outvoted. Experiment 4 varied the regularisation parameter C of the SVC range of values (0.1~10.0), to see if the default value of the margin penalty trade-off did not perform well. These, combined with the differences in feature representation, model combination, weighting and sensitivity to hyperparameters, discussed in Section 5.2, provide viewpoints of the practical level limit on performance.

3.8 Transformer-Based Approach

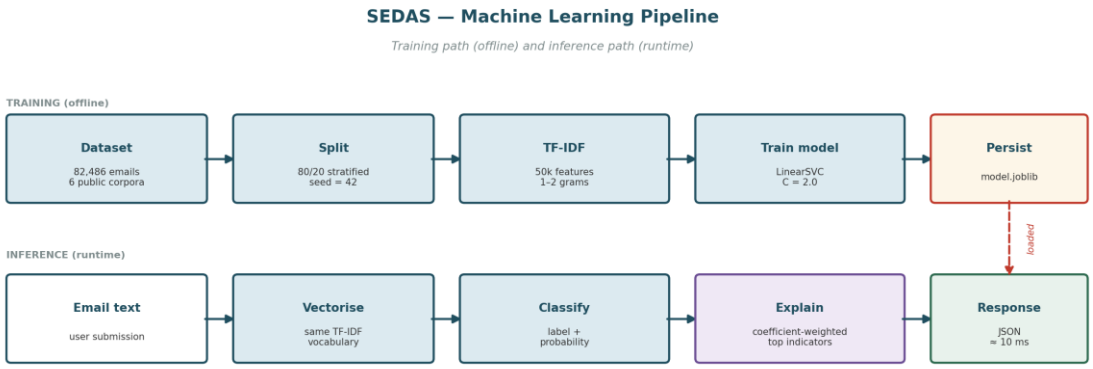

Figure 3: Machine learning pipeline — offline training path and runtime inference path.

A completely different representation is being tried out: TF-IDF is supposed to store a collection of words as a message, removing the order of the words; transformers will store each word as part of a sequence, which means they model different word-sense occurrences of the same word, depending on the neighboring words. DistilBERT was chosen as a more efficient (and smaller) BERT that is appropriate for the CPU—only hardware. The random state for stratified sampling was 42, there were 6000 emails sampled (3120 phishing, 2880 legitimate), 128 sequences of the tokens in each email, batch size 8, two epochs and a learning rate of 2e-5. These configurations trade-off on context size vs. the amount of CPU resources that can be dedicated to executing the transformer, and in these configurations, the transformer has yet to be trained on the complete corpus (which accounts for roughly 7.3% of the full corpus), and full-data fine-tuning will be required and deferred to future work for GPU resources.

3.9 Refinements from the Part A Design

Table 2: Refinement register — changes from the Part A specification with justification.

Ref	Refinement	Justification
R-01	Ethereum smart contracts → SHA-256 hash-chained registry	Preserves tamper-evidence without gas fees, wallet infrastructure or block latency; Ethereum retained as future work
R-02	Browser extension → web form and API	Extension store publishing and cross-browser testing exceed the timeline; verification capability fully demonstrated
R-03	Live LMS/SIEM/SSO integration → documented API contracts	No institutional access available to a student team
R-04	F1 ≥ 0.95 target → honest gap analysis approach	Model quality depends on data volume and compute unavailable in the unit
R-05	Latency < 200 ms (GPU) → < 2,000 ms (CPU)	GPU compute out of budget; latency measured and reported so the production path is evidenced
R-06	Kubernetes and Kafka → direct REST between services	Scale-out infrastructure does not change functional behaviour at MVP scale; microservice decomposition retained
R-07	Two institutional case studies → single simulated scenario	Real pilots are outside unit scope and would require ethics approval
R-08	SMS/IM interception → documented future work	Separate delivery channel with material engineering cost

The changes in the Part A design were intentional scope changes that occurred during Construction, (Table 2 above), and were not a "change of direction" at the point of implementing the design changes. The most critical of this, R-01, secured Ethereum smart contracts and opted for the blockchain-less, SHA-256 hash-chained registry, meaning the system was no longer decentralised as decentralised consensus was never required for tamper-evidence, nor would there be any gas fees since the system was not based on smart contracts, and no wallet infrastructure would be required since registry itself was not blockchain-based. Measured latency was still in the single and low double digit MS range—a lot lower than the re-baselined re-target of < 2,000 MS for commodity CPU based hardware. Refining infrastructure ambition did not reach a stage of assessing infrastructure behaviour (overall).

3.10 Evaluation Protocol

Table 3: Evaluation protocol.

Parameter	Value
Split	80/20 (Training on 80% of the examples, testing on 20% of the held-out examples (stratified by class))
Training set	65,988 emails
Test set	16,498 emails (not seen during training/completing task)
Random seed	42 (initially were random, but were set to the same value here to have a complete repeatability)
Metrics	Accuracy, precision, recall, F1, false-positive rate, confusion matrix, latency.

All classic experiments employed an 80/20 stratified hold-out design (65,988 training and 16,498 test) since it is important not to measure memorisation but generalisation if one measures on the training data. The ratio of phishing to legitimate stays constant in both partitions by using the same fixed, random seed; a fixed random seed of 42 has been used for the allocation, so the amount of data in each category — and, consequently, comparisons of the various models — can be independently checked. Evaluation based upon accuracy, precision, recall, F1, false-positive rate, confusion matrices and measured latency: F1 is a mainline evaluation measure as it is a measure of choosing the option that is best for the overall model even when it comes at the cost of the other measure; false-positive rate is reported separately as there is a direct deployment cost to unnecessary blocking; latency is reported as a measure of suitability for use in interactive use.

4. Implementation and deployment of the system

The base paper's Section 4 covers web deployment of its model. This section covers SEDAS's implementation, which extends beyond a single deployed classifier to four integrated components.

4.1 Detection Service

Table 4: Detection service endpoints (port 8001). Measured latency ≈ 7-11 ms.

Endpoint	Method	Behaviour
/classify	POST	Accepts email text and returns a predicted label, risk score (0-100), top contributing indicators, and measured latency.
/health	GET	To get the service status (for monitoring purposes)

Detection Service is an application comprising of a FastAPI service deployed on 8001. It features endpoints like /classify, which are supplied e-mail content and returns the forecasted label, a phishing danger rating (between 0 and 100), the top indicators for the label forecast, and the measured latency and monitoring can be performed via the endpoint /health. Explainability - is parallel to the trained linear model: the message to explain is vectorised as it was when training the linear model (the same TF-IDF vectoriser is used); each of the features is multiplied by the coefficient for that feature from the trained linear model; the top 5 features (those which contribute the most positively) will be returned as indicators. It does satisfy level FR-02: Words in the message to which the person seeing the words VERIFY, ACCOUNT or IMMEDIATELY (flagged) can associate the words and phrases. The measured latency was of the order of ~7-11 which is well below the less than 2000ms target set by the NFR-03.

4.2 URL Verification Service

Table 5: URL heuristic indicators (port 8002). Measured latency ≈ 1.4-9.3 ms.

Heuristic indicator	Rationale
No HTTPS	Unencrypted transport for a credential-collecting page
Raw IP address as host	Legitimate services almost always use domain names
'@' in URL	Everything before '@' is discarded by browsers, hiding the true destination
Multiple hyphens in domain	Common in lookalike domains imitating a legitimate brand
Excessive subdomain depth	Places a trusted-looking name in a subdomain of an attacker domain
High-risk TLD	Certain TLDs are disproportionately represented in abuse data
Known URL shortener	Destination cannot be inspected before clicking
Abnormal length	Long URLs obscure the destination in address bars
Credential-lure keywords	Terms such as login, verify, secure, account, password

The URL verification service consists of two layers that are used in conjunction to verify the URL. The first instantly conducts structural analysis to search for parameters such as the absence of a secure https as well as simple IP host, “@” obfuscation, numerous subdomains, high-risk TLDs and recognized credential-lure words, and offers threat score and easily understood outcomes even when checking a never-seen URL. The other

repository is an append-only trust registry where the content of each record on the repository is used to calculate its SHA-256 hash along with the previous record's at the bottom of the repository, tampering of any previous record is going to break this link, and enable GET /chain/validate to check the complete sequence, and find out if it was tampered with. The verdict resolution also incorporates both layers (previously-reported malicious URLs return blocked irrespective of their current score achieved through the heuristic process) and represents an approach to tamper evidence rather than a decentralised consensus, such as MLPhishChain's leverage of Ethereum blockchain [17] that dispenses with any wallet infrastructure, which, owing to the MVP scope, foregoes gas charges. The average verification latency of measurement was around 1.4 – 9.3 milliseconds.

4.3 Awareness Training Service

Users sign up as a persona (either student/staff/administrator) and lessons are geared towards a particular persona or different roles. In line with evidence that feedback timing plays a crucial role in the potential behavior-changing effects of training [4] the quiz endpoint does not show the answer or explanation until submitting the answer, and then immediately provides the answer and explanation on a per-question basis. Risk is re-calculated with an exponentially-weighted moving average (EWMA) after each attempt with a weighting factor of 0.4, making for a smooth weighting of risk with recent work influencing the risk more than older work, but no one result dominating the risk. Risk bands are low – below 33; medium – below 66 and high – at or above 66. There is an intentional polarity between the difficulty of the lesson content and eligibility for risk such that high-risk users get foundational lessons whereas low-risk users get more difficult lessons: a _fallback_ mechanism is used if a lesson does not exist at the desired difficulty level itself, searching multiple lessons or alternative lessons within that same tier (FR-05, FR-06) (content is append only, failure is remedial).

4.4 Interface and Governance Layer

The browser interface offers one demonstration surface for all the three procedures, and shows the detection outcomes, the outcomes found for URLs and the training progress. To create the interface, it was decided to respect the advice of the supervisor, that the web development should not be in charge of the execution and evaluation of the machine learning and should therefore, take a supporting role. Governance layer only contains what was partially captured: append only attempts, risk scores, lesson completions and registry records are gathered in structured form for future use in querying, and the production dashboard and role based access is still some work to be done (FR-07 partial).

4.5 Technology Stack

Table 6: Technology stack by layer. No component requires a GPU.

Layer	Technology	Purpose
Runtime	Python 3.14	Application language across all services
Web framework	FastAPI + uvicorn	REST endpoints, validation, automatic OpenAPI generation
Validation	Pydantic	Typed models; rejects malformed input before handler execution
Machine learning	scikit-learn, xgboost	TF-IDF, LinearSVC, RandomForest, MultinomialNB, LogisticRegression, XGBoost
Transformer	PyTorch, Hugging Face Transformers	DistilBERT fine-tuning
Persistence	SQLite 3, joblib	Service datastores and model serialisation

Layer	Technology	Purpose
Integrity	hashlib (SHA-256)	Registry chain hashing
Frontend	HTML, CSS, JavaScript	Demonstration interface; no build step
Testing and CI	pytest, GitHub Actions	23 automated tests, executed on every push

4.6 Security Controls

SEDAS was evaluated against the OWASP Top 10 (2021) categories, by manually reviewing all the sources against the automated test suite. The severity is considered to be High, Medium, Resolved (implemented and verified by a test) or Not Applicable (Table 7).

Table 7: Definitions of terms used in the rating of severity for this assessment.

Rating	Meaning at this project's scope
High	The attacks can be accomplished easily without special privileges; data exposure, data integrity loss or denial of service could cause significant and material impact. Should be able to veto the choice to go into production.
Medium	Exploitable, but in need of some condition and/or with a limited effect. Acceptable as MVP/demonstration with the issue being noted but it should be addressed prior to being used in production.
Resolved	Category has been evaluated and there is a control and it has been validated using automated test.
N/A	A system's architecture for which the classification does not apply, justify why.

Table 8: Summary on all ten categories of OWASP — (Three are High, 5 are Medium — 1 Resolved and 1 Not Applicable).

Category	Severity	One-line status
A01 Broken Access Control	High	No authentication or RBAC implemented
A02 Cryptographic Failures	Medium	No TLS, no encryption at rest — acceptable at local-dev scope only
A03 Injection	Resolved	Parameterised queries; verified by adversarial test TS-01
A04 Insecure Design	Medium	Lacks any formal threat model, or rate limiting
A05 Security Misconfiguration	Medium	Permissive CORS + API docs unrestricted
A06 Vulnerable Components	Medium	No automated dependency vulnerability scanning
A07 Authentication Failures	High	There is no authentication mechanism in place (related to A01)
A08 Software/Data Integrity	Medium	Model file doesn't have an integrity check for loading into the registry; integrity check for the hash chain in the registry has been resolved.
A09 Logging & Monitoring	High	Lack of basic security relevant audit logging, other than basic health checks.
A10 SSRF	N/A	A design that doesn't make any request to an external server from the back-end logic.

4.6.3.1 Overall posture

Table 8 summarises all 10 categories and shows that there are actually only three High (Broken Access Control, Authentication Failures, and Logging and Monitoring Failures), based not on the finding itself (which are independent), but from Table 8 they share only one root cause: No Production Authentication at MVP scope (FR-08 deferred). An injection marking of Resolved indicates that injection is independently verified (as opposed to being simply reviewed) by an adversarial test (marking of TS-01). The applicability, evidence to support each category and the action to be taken in case of violation are detailed in a separate OWASP Security Assessment on a per category basis. Table 9 presents the remediation actions by the severity that they were addressed (not necessarily in the order that they were implemented).

Table 9: Remediation actions ordered by severity addressed, not necessarily by implementation order.

#	Action	Addresses	Effort
1	Use authentication along with role based access control (institutional SSO as desired state for the future)	A01, A07	High — future work
2	Extensible security event logging (validation failures, access attempts when resources are unknown, reports submitted)	A09	Medium
3	Only allow CORS from sources that are known to you; Do not allow CORS /docs, and /openapi.json outside of development	A05	Low
4	Implement rate limiting for write operations to POST /report (and all write operations)	A04	Medium
5	Add pip-audit / Dependabot to the CI pipeline	A06	Low
6	Add a checksum verification step before loading model.joblib	A08	Low
7	Feasibility of terminating TLS prior to deployment to the network	A02	Medium — deployment-time

The level of effort needed to accomplish remediation, ranging from easiest to most difficult, is presented in the order listed in Table 9: (restricting CORS, adding dependency scanning and checksumming of the model file before properly-scoped future work (authentication, logging and rate limiting).

4.7 Testing and Quality Assurance

The consolidated test matrix (Table 10) has 23 tests all of which passed.

Training and references for training suite: 100% coverage of documents 100% of references covered.

Suite	Test IDs	Coverage	Requirements
Detection	TD-01.TD-05	Health, phishing detection, legitimate pass-through, explainability, latency	FR-01, FR-02, FR-09, NFR-03
URL verification	TU-01.TU-03	Heuristic flagging, report-then-blocked with proof, chain integrity	FR-03, FR-04
Training	TT-01.TT-06	Scoring, EWMA risk, banding, adaptive selection, fallback	FR-05, FR-06
Security	TS-01.TS-06	Injection, malformed input, unknown resources, answer confidentiality, health, chain integrity	NFR-05

4.7.1 Testing strategy

For each module (TD detection, TU URL verification, TT training, TS security), we have tests (total 23) which are stable in identifier, and are organised in a layered way, which is traced back to the requirements. Both the success variants and failure variants are tested – (negative testing – malformed data, unknown resources, injection like values), is important as a suites that only tests the “happy path” reveals little about behaviour under misuse. The suite is executed on every push with GitHub Actions, and it executes without problems missing the locally trained model when it is not present in version control (which would be for model-free suites), but for those requiring the model it does run as well without any hiccups.

4.7.2 Security testing

A set of executable evidence is used to validations the controls in the security suite. TS-01 posts a URL with SQL metacharacters, and makes sure the registry is still up and the hash chain still matches — thus, parameterised queries did not execute the SQL provided in the post. TS-04: Requests a quiz, and states there is no right answer or explanation in the answer - i.e., no answer key retrieval pre-submission. TS-06 incorporates several entries to the registry, and then revalidates the entire chain with the assumption that the relationship of the hashes would remain constant with repeated writes – not just for a single record. When combined these allow for some tangible proof of persistence, secure disclosure and tamper evident.

4.7.3 Defect identified through testing

When increasing the number of tests from 8 to 23 tests, it turned out that there was a real defect: both the URL verification and the training service had a module called serve, Python's module cached the first of which was loaded and it would silently hit training service and fail with HTTP 404. The failures were found by using the same pattern of explicitly being able to import by file path, as the passing security suite does, in place of by name for the url suite (imported via importlib). This is an example of using test as a discovery process, not a "confirm" process; even though the services are

running in different processes, they could name their modules the same and cause silent coupling in the shared test process.

5. Results and Discussion

This section will show the result or outcome of the study that was gathered or acquired through an empirical study. We tested six types of classifiers on the same e-mail data set as in the base paper [1] which is a total of 82,486 e-mail messages of which 65,988 are used for training and 16,498 for testing, both sets being 80/20 stratified by class, using a fixed random seed of 42. Three of them are reproductions of the ones assessed in [1] while two of them are traditional extensions, one of them is a preliminary scale assessed transformer. Four more so-called optimisation experiments sought to find out if you can still surpass the benchmark which was replicated.

5.1 Benchmark Results and Operational Metrics

Table 11: The result of the F1 performance of all classical method vs the base paper's performance.

Method	SEDAS F1	Paper [1] F1	Difference	Evaluated in [1]
SVC / LinearSVC	99.16%	99.0%	+0.16	Yes — best in [1]
Random Forest	98.71%	98.0%	+0.71	Yes
Logistic Regression	98.65%	—	—	No — extension
XGBoost	97.56%	—	—	No — extension
Multinomial Naive Bayes	97.47%	98.0%	−0.53	Yes

Table 12: Five fold stratified cross validation, mean ± standard deviation, seed=42) confirms that the numbers given in the single-split case in Table 11 are good cross validation numbers. One can compare SVC and its next best competitor (Random Forest) with the help of paired t test; in this case, there are five folds t = 16.60 and p = 0.0001..

Method	Mean F1 (5-fold CV)	SD	Single-split F1 (Table 11)
SVC / LinearSVC	99.233%	±0.086%	99.16%
Logistic Regression	98.724%	±0.091%	98.65%
Random Forest	98.705%	±0.077%	98.71%
XGBoost	97.588%	±0.074%	97.56%
Multinomial Naive Bayes	97.495%	±0.129%	97.47%

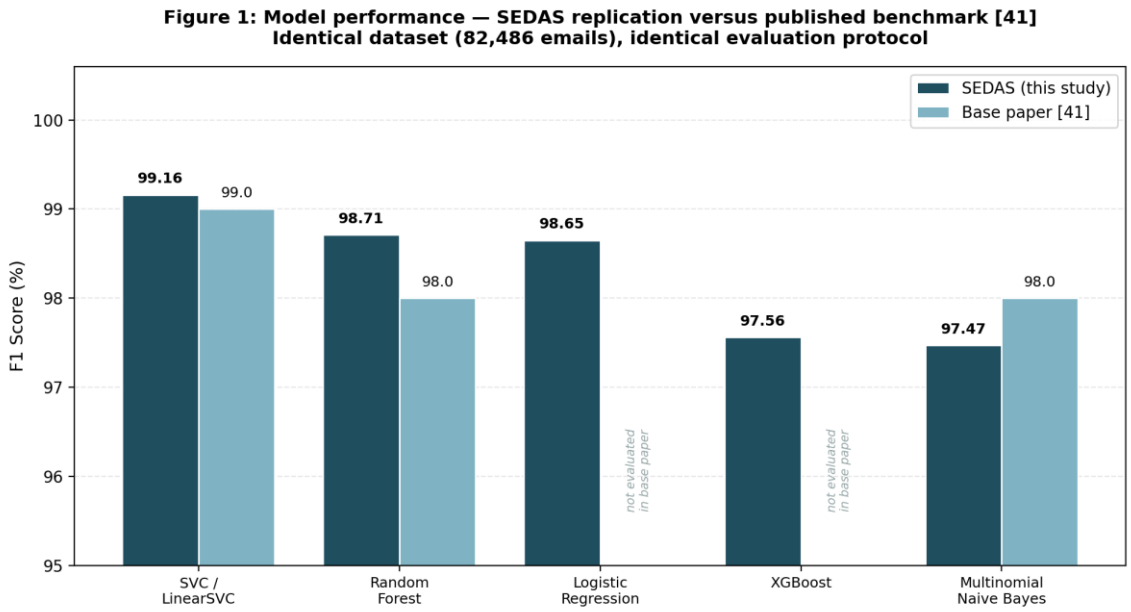

Figure 4: Model performance — SEDAS replication versus published benchmark [1].

The best classical method is the LinearSVC, which, as mentioned in the base paper, was able to achieve a F1 score of 99.16% whilst the published F1 was 99.0%. Random Forest obtained 98.71% accuracy as compared to 98.0% accuracy of Multinomial Naive Bayes. These differences reflect the fact that the best method is SVC and are not evidence of superiority as [1] does not show fold-level results or as much detail about the preprocessing as SVC. Both the Logistic Regression with 98.65% accuracy and the XGboost with 97.56 % accuracy are not the best result in the base paper, therefore the replication doesn't have any contribution in itself, but serves as a good base line.

Table 13: Metric comparison for SVC and Random Forest.

Metric	SVC (ours)	SVC [1]	RF (ours)	RF [1]
Accuracy	99.12%	99.1%	98.66%	98.4%
Precision	99.13%	99.0%	98.80%	98.0%
Recall	99.18%	99.0%	98.62%	99.0%
F1	99.16%	99.0%	98.71%	98.0%

Table 14: There is a trade off between precision-recall, but this may be acceptable with regard to the Multinomial Naive Bayes.

Metric	MNB (ours)	MNB [1]	Difference
Accuracy	97.41%	97.8%	−0.39
Precision	99.06%	97.0%	+2.06
Recall	95.93%	99.0%	−3.07
F1	97.47%	98.0%	−0.53

Table 15: Full metrics for the two extension methods, not evaluated in [1] and therefore without a paper-comparison column

Metric	Logistic Regression	XGBoost
Accuracy	98.59%	97.43%
Precision	98.67%	96.32%
Recall	98.62%	98.83%
F1	98.65%	97.56%

Figure 2: Metric-level comparison across the three replicated methods

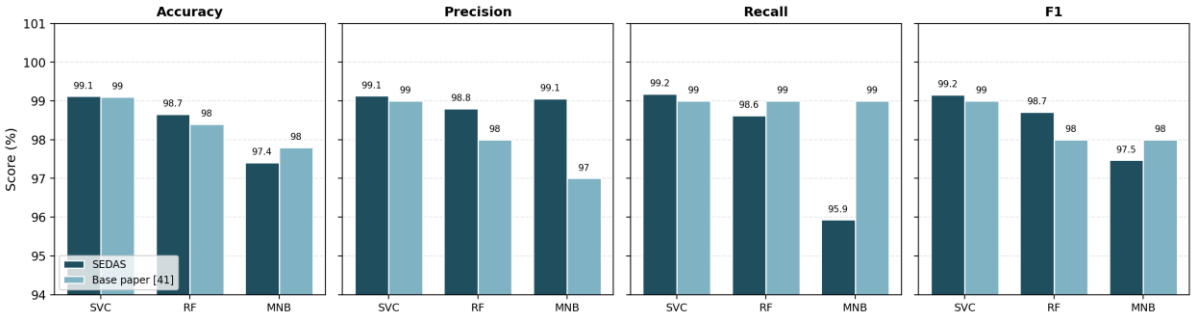

Figure 5: Multiple-metrics are used to do metric level comparison for the three replicate methods.

Figure 3: Confusion matrices on the 16,498-email held-out test set

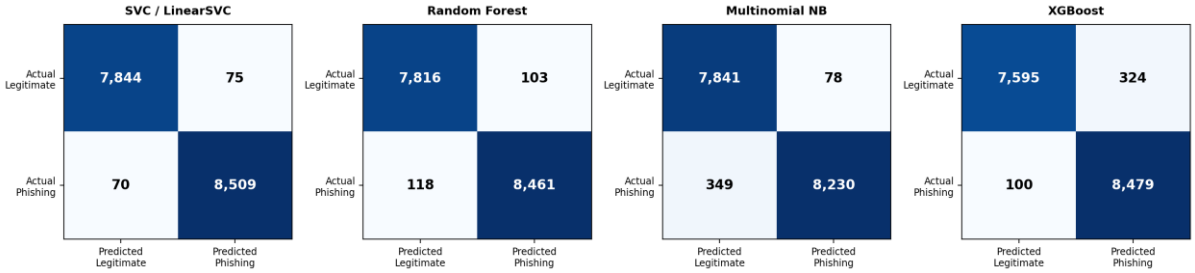

Figure 6: The confusion matrices for email test set (held out) 16498_email are shown.

Detail is provided at the metric level by some of the figures. However, Naive Bayes shows a high precision of 99.06% but the recall rate of 95.93%, this implies that Naive Bayes has 349 false negative output whereas SVC has 70 false negative, this is because of their conditional independence assumption which does not offer them any means to utilize multi-word combination of the phishing text. With XGBoost it's the other way around with 324 false positive risk flags for each 100 false negative risk flags. The total number of errors (145), turned out by the SVC with the most balanced profile should be the smallest.

Table 16: Table shows the number line of distribution of the errors occurred in held out test set that had 7,919 genuine and 8,579 phishing samples. In [1] all the above figures are omitted (only there are quotes).

Method	False positives	False negatives	FP rate	Total errors
SVC / LinearSVC	75	70	0.0095	145
Random Forest	103	118	0.0130	221
Logistic Regression	114	118	0.0144	232
Multinomial Naive Bayes	78	349	0.0098	427
XGBoost	324	100	0.0409	424

Figure 4: Error-type breakdown — operational metrics not reported in the base paper [41]

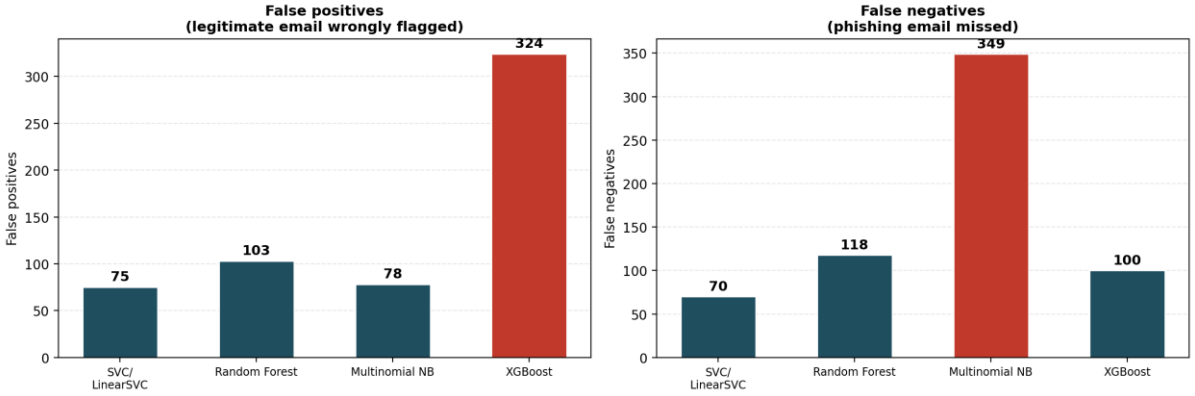

Figure 7: Error-type breakdown — operational metrics not reported in the base paper.

These operational metrics (also for the false positive rate and number of absolute errors) are a contribution over cluster results from [1] which only report the aggregate classification metrics (full confusion matrices). Not certain about whether they have justifiable pros and cons, but equivalently, XGBoost has an unrealistic high indicator for False Positives (4.09%) and Naive Bayes, it has a high value when it comes to attacks missed (349) but a low one when it comes to False Positive (0.98%). SVC rejects both failure modes, and can decide on model selection that will lead to consequences that are institutional more so than based on F1 alone.

5.2 Extension Experiments and the Performance Ceiling

Table 17: Results of four optimisation strategies applied beyond direct replication.

Experiment	F1 Result	Change from baseline
SVC baseline (replication)	99.16%	Reference
SVC + engineered features	99.16%	+0.00 — no change
Equal-weight voting ensemble	99.13%	−0.03 — slight degradation
Weighted voting ensemble (SVC ×3)	99.17%	+0.01 — negligible
SVC hyperparameter tuning (C = 2.0)	99.17%	+0.01 — negligible

Table 18: Table shows the results of the five folds cross validation for the four different optimisation strategies: Paired significance testing of data on each fold cross validation with the data from the SVC baseline model (significance level of α = 0.05%). Note that the results of the weighted ensemble and SVC tuned (C=2.0) are not means - rather they are the same results over the individual folds.

Configuration	Mean F1 (5-fold)	SD	vs baseline (p-value)	Result
SVC baseline	99.233%	±0.086%	—	Reference
SVC + engineered features	99.224%	±0.092%	p = 0.122	Not significant
Equal-weight ensemble	99.112%	±0.069%	p = 0.0008	Significant degradation
Weighted ensemble	99.217%	±0.118%	p = 0.314	Not significant

SVC tuned (C=2.0)	99.217%	±0.118%	p = 0.314	Not significant

Table 19: The parameters that are searched for SVC are shown. The optimum value is 0.018 points greater than default and they should be between 0.285 F1 points more or less.

C value	F1 Score	Note
0.1	98.888%	Under-regularised for this data
0.3	99.097%	
0.5	99.115%	
1.0	99.155%	Default value used in replication
2.0	99.173%	Best observed
5.0	99.114%	
10.0	99.103%	Diminishing returns

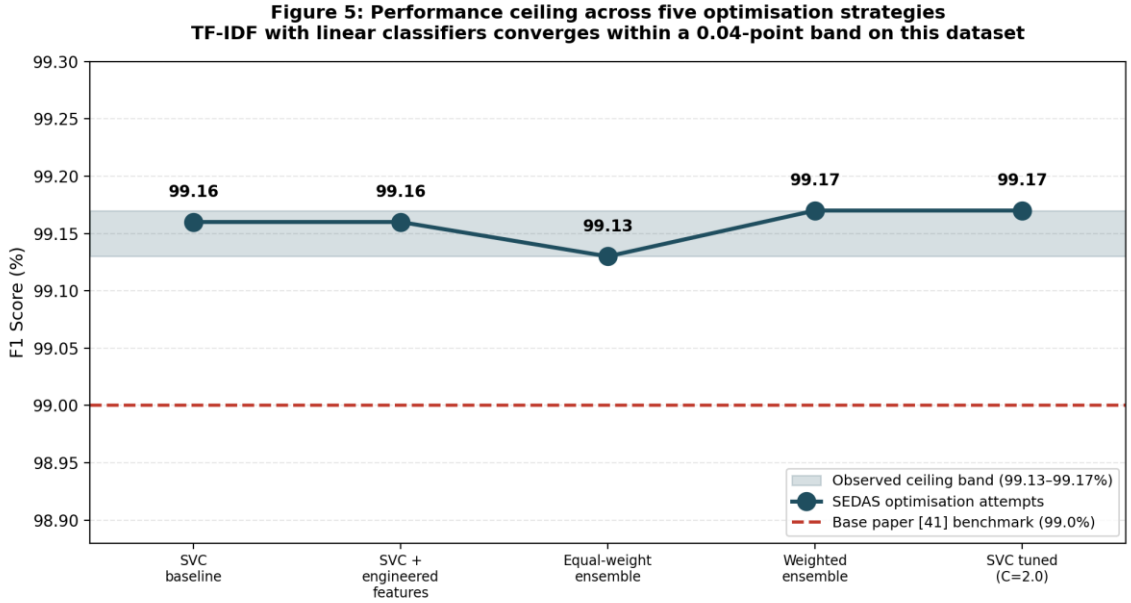

Figure 8: The results from a series of five performances differing degrees of performance optimisation are the highest (the most demanding one is depicted).

Four different Optimisation Strategies are compared with the 99.16% SVC but none of the figures can be considered significantly better. Likewise, when compared with the others 3 feature engineering, equal weight ensemble was the only one that was significantly worse (p=0.0008) while the other others were not significantly different from each other (engineered features p=0.1217, weighted ensemble p=0.3139, tuned SVC p=0.3139). All of the results have small magnitude (0.04 point). Using the base model (TF-IDF + linear SVC), a good work can even be done in this task..

TF-IDF already does consider the discriminating features themselves through the features they engineered (URL count, capital ratio, keyword counts) so it didn't offer any signal. This situation occurs when one vote is given to SVC, in such case it will be considered that both the learners are agreed, the ensemble will be worse and the weaker learner (Random Forest) will overpower the learning of SVC. Thus, there isn't any automatic answer, just a better out of two modes which might be achieved by awarding the majority voting.

It was at this point that both models (SVM and weighted ensemble (3:1:1)) matched at that point on each of the folds they achieved equal F1 score as at this point you cannot out-vote two '1' voting models (SVM) with three '1' voting models (SVC) using hard voting. The Hyperparam tunning Results were similar with best hyperparameter being 0.018 F1 higher than default(that is max degree equal to 3) and all the C values were within a range that is 0.285 F1. This is not a case of 4 failure attempts that don't seem to have any connection, it's a discovery that will aid in future actions. This was thought to be the motivation for the following experiment: it is much more likely that some real improvement can be made before (and not as part of) this "full up" decision space is further optimised.

5.3 Transformer Comparison and Precision-Recall Trade-off

Table 20: The basic settings of the DistilBERT experiment are given.

Parameter	Value	Rationale
Model	distilbert-base-uncased	CPU-feasible variant of Distilled BERT, which is a smaller & faster version of BERT with to all intents and purposes the same performance.
Training sample	6,000 emails (7.3% of corpus)	CPU-only hardware constraint
Class balance	3,120 phishing / 2,880 legitimate	Stratified from the full corpus
Train / test split	4,800 / 1,200	80/20, stratified, random_state 42
Max sequence length	128 tokens	Balance of context and compute
Batch size / epochs	8 / 2	Memory and time constraints
Learning rate	2e-5	Standard for transformer fine-tuning

Table 21: DistilBERT was used to get the results for the preliminary evaluations.

Metric	Value	Note
Accuracy	97.92%	On 1,200 held-out emails
Precision	97.77%	Of emails flagged phishing, proportion correct
Recall	98.24%	Of actual phishing, proportion caught
F1 score	98.00%	Headline comparison figure
False-positive rate	2.43%	Higher than all classical methods except XGBoost
Confusion matrix	TN=562, FP=14, FN=11, TP=613	25 total errors from 1,200 test emails

Table 22: All six evaluated methods. The DistilBERT row is not directly comparable, having been trained and tested on a subsample.

Method	F1	Training set	Basis of comparison
SVC / LinearSVC	99.16%	65,988	Full dataset
Random Forest	98.71%	65,988	Full dataset

Method	F1	Training set	Basis of comparison
Logistic Regression	98.65%	65,988	Full dataset
DistilBERT (preliminary)	98.00%	4,800	7.3% subsample — not directly comparable
XGBoost	97.56%	65,988	Full dataset
Multinomial Naive Bayes	97.47%	65,988	Full dataset

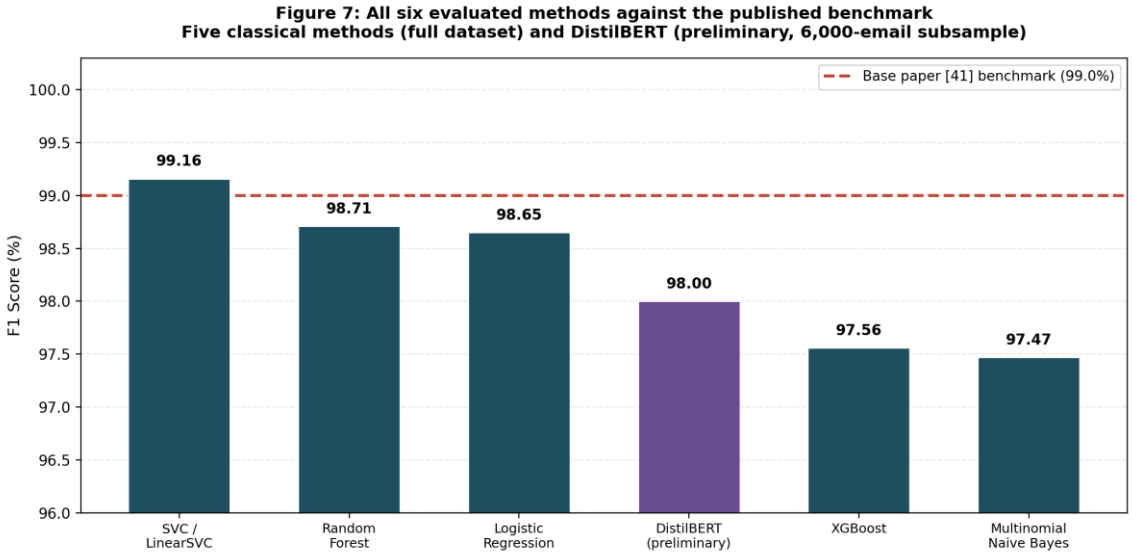

Figure 9: All six evaluated methods against the published benchmark.

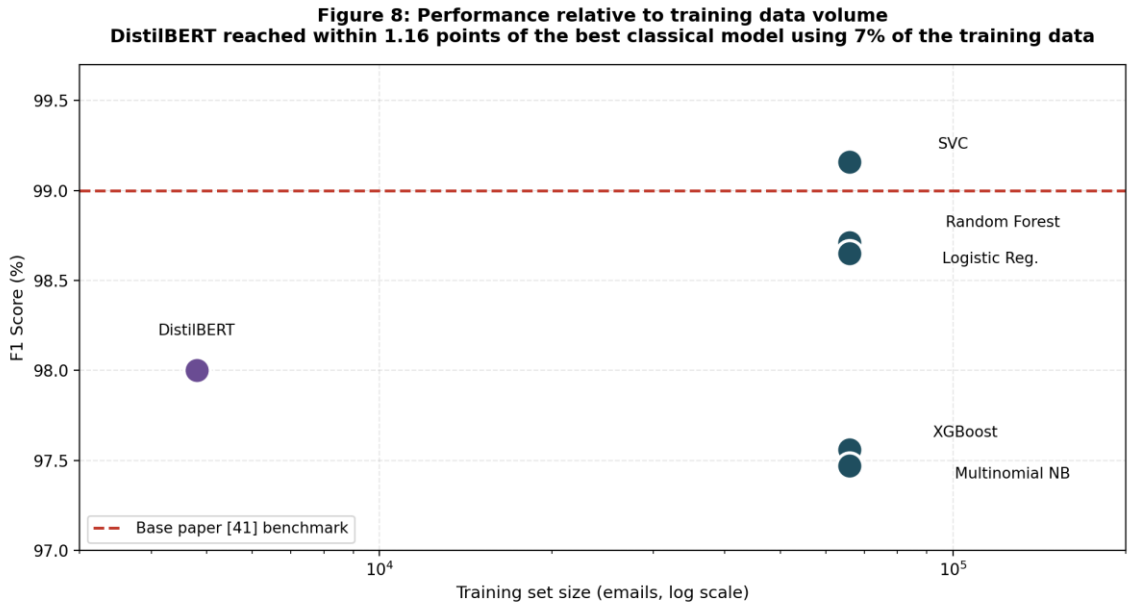

Figure 10: Performance relative to training data volume, logarithmic scale.

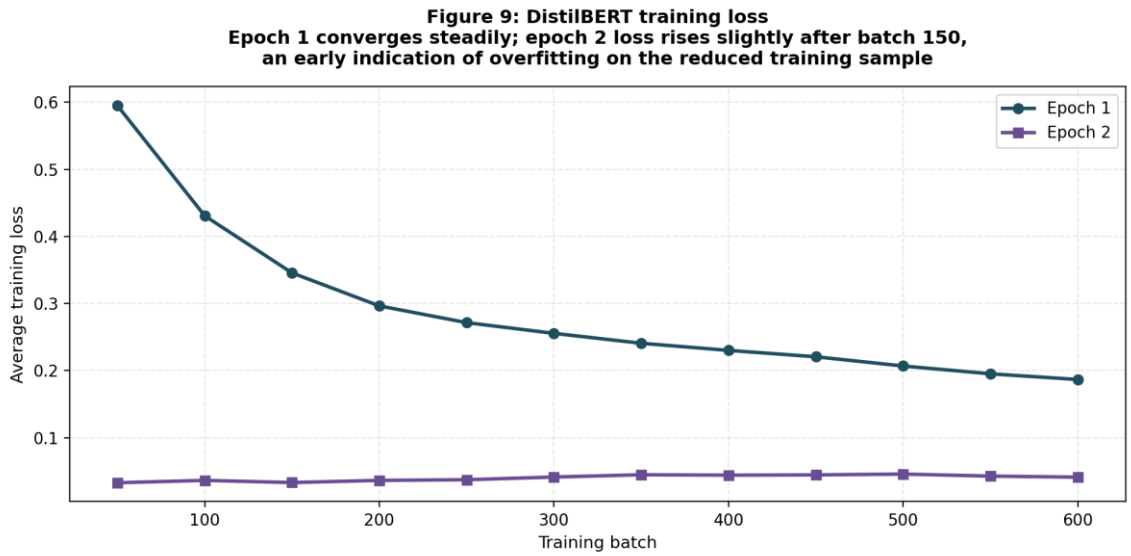

Figure 11: DistilBERT training loss across both epochs.

With a portion of the email dataset they managed to reduce to 6,000 emails (4800 training / 1200 test) they distilled DistilBERT on a CPU, to check if there could be a contextual representation of a text acting as a replacement for TF-IDF. It achieved 97.92% accuracy, 98.00% F1, with a 2.43% false-positive rate. I believe that this should be a test for technical feasibility, and of data-efficiency, but NOT for whether or not transforms are closer to the classical ceiling, since classical can't be compared to DistilBERT's data point (this is just off the top of my head) of 4800 e-mail.

DistilBERT achieved the best results of all the models, with a score within 1.16 F1 points of the score of full data SVC, which is about 7% of the training corpus. This can be explained by the fact that the training loss is continuously decreasing between the first and second epochs (0.60 > 0.19), which is the first sign of "over-fitting" with the smaller size of the sample: our goal with Section 6.2. is to run a full-corpus evaluation on the GPU.

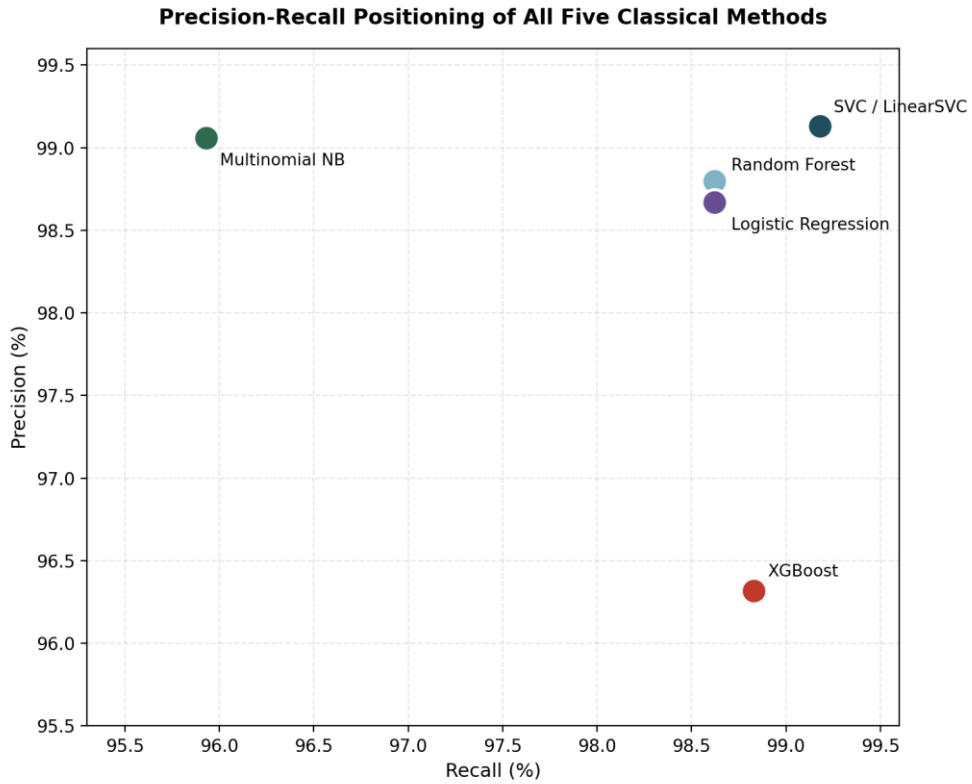

Figure 12: Precision-recall positioning of all five classical methods.

LinearSVC has best performance in precision / recall space as none of the dimensions (precision, recall) is given up (99.13% precision, 99.18% recall). Unlike the Naive Bayes, which has a conservative vs. liberal configuration (high precision, low recall vs. high recall, low precision), XGBoost is in somewhere in the middle. For those that are deployed in institutional settings the choices of balance to use can be narrowed down to using SVC's balance which aims to minimise the total error, and has a recall of over 99%.

5.4 Significance and Limitations

The study is accompanied by four strands that expound on the importance of the study. Reproducibility, for example: Data sets are small, and the methodology isn't that well documented, meaning that there is some modelling aspect that is re-runable (82,486 emails in a data set; with our seed, and our set-up has been revealed). Secondly, based on the 'performance-ceiling finding' it is not necessary to investigate this space of the 4 independent optimisation strategies to the extreme end as these have all collapsed to the TF-IDF/classical setting. Third and finally, all these measurements don't reflect the full picture, e.g., all 424 errors (Naive Bayes) vs all 427 errors in the other direction with a higher cost (XGBoost) – which was only reflected by F1. Fourth, integration: social engineering cannot be all about the classification alone - the deployed SEDAS provider might be a classifier deployed onsite, integration with a rule/sensor might provide some part of the defence in depth, but as explained above, in higher education it may be so that multiple different consumers (and distributed) might need some part of the defence in depth to be human.

Limitation of scope of claim is evident. When a data is part of the corpus, it is a public research data and actual rather than real data from the institutions, therefore, domain shift is not tested. More experiments have been conducted at SEDAS (Sections 5.1-5.2) but a significance test has not yet been done with respect to [1] itself as they do not publish their data at the fold level. Quite a bit of finetuning of extension strategies and DistilBERT

seemed missing. However, it was not an 'authentic' community because it was not a community at a behavioural level, rather it was at a technical level. No MVP scope authentication, no access control and the hash-chain URL-registry wasn't distributed at all, just the most basic one that would work for any MVP – in the successful experiment it was CORS was also set up as way too permissive.

5.5 Outcomes Against Objectives

Table 23: Outcomes, and table of Objectives (part A)..

Objective (Part A)	Target	Achieved	Status
Detection F1	≥ 0.95	0.9917 (tuned SVC)	Exceeded
False-positive rate	< 2%	0.95%	Exceeded
Classification latency	< 2,000 ms	≈ 7-11 ms	Exceeded
URL verification latency	< 2,000 ms	≈ 1.4-9.3 ms	Exceeded
Registry tamper-evidence	Verifiable chain	Chain validation confirmed	Met
Adaptive training	Persona-based delivery	Implemented and verified	Met
Risky-click reduction	30% (simulated)	Not measured	Deferred

As seen on the Table 23, the project has fulfilled most of the measurable Part A sub-goals including 0.9917 achieved towards detection F1 while the target was 0.95, false positive is 0.95% which is below the target line of 2%, latency both sub-goals are met with significant margin. Because of the only one not met score - the 30% reduction on "Risky clicks" which cannot be performed even with an ethics approved study involving real subjects will be carried over.

6. Conclusion and Future Work

6.1 Conclusion

We have adopted the approach of Al-Subaiey et al., [1] which provides a very good score for the machine learning based detector: (a) F1 score is 99% with regards to the machine learning based approach of the phishing defence problem, (b) F1 score is 99% for the more general approach that is based upon social-engineering. Not only has SEDAS reproduced the process on the same massive volume of 82,486 emails, but they have created many improvements – an integrated defence architecture, more in-depth metrics in operation and controlled testing of optimisation which have been added to the mix. In terms of replication, the figure of the LinearSVC was 99.16% which was almost the same as the published figure (99.0%) and the other models, Random Forest and MultiNB maintained the same respectively. What is more important, however, is that another 4 optimisation strategies were tested and were significantly worse (all of them lie in a very small range around this one) and that the one given in the results is the set up of the TF-IDF/classical pipeline (under evaluation) tested near the practical limit within this corpus.

When it comes to the overall number of errors (false positives and false negatives) that operational analysis confirmed that assessment: SVC ended up with the lowest number of errors (70 FP), while XGBoost confused too many attacks and Naive probably failed to flag enough and was relatively under none of the provided choices. In the initial DistilBERT study, it was possible to model the context with an accuracy of 98.00% F1 and only 4,800 emails have been trained – but this should not be used for comparison. These target to the following explanations for questions guiding the project: Benchmark reproducibly, Technically nothing is more basic than benchmark – improvements on classics have been tried, if technically possible then project is also technically possible, in the future – explanations (classification), check of urls, training/governing of steering data in one system.

It's not just the Blastoff - score points game, it's process-discovering of how you can operate your mind and others (like the iPhone psychology-minefield) and how you can do an operation through evidence and have a model of the technolutions and actual social engineering process that you can likewise operate.

6.2 Future Work

In the future it would be nice to see if the classical ceiling is due to the TF-IDF representation or due to some other issue in case of running DistilBERT (and even BERT, RoBERTa and other bigger models as suggested in [9] and [3]) on the entire corpus. It may also be worth playing with as an additional part – the classical extension space can be profited and potentially it may be worth playing around – whether increasing the weak signals from sparse features (with XGBoost etc) is a value-while is worth investigating. Next, production security, where FR-08 “institutional SSO/role based access control” adds production security – rate limiting on the URL report endpoint and attribution will only be able to respond from the cross origin domain or if they come from a whitelisted origin (OAuth only). With a real world (institutional) pilot (and ethics approval) it would be possible to measure how much risky-clicking behaviour reduction and training retention is possible in the long term especially in the context of the new and emerging threat of AI-generated phishing [6] [7]. These extensions could be added on to the MVP, which has been validated, to create a platform which can be assessed from a behavioural point of view and is production ready.

References

APA 7th edition, continuing the Part A reference register [1]-[40].

[1] Al-Subaiey, A., Al-Thani, M., Alam, N. A., Antora, K. F., Khandakar, A., & Zaman, S. M. A. U. (2024). Novel interpretable and robust web-based AI platform for phishing email detection. Computers and Electrical Engineering, 120, 109625. https://doi.org/10.1016/j.compeleceng.2024.109625 [Q1]

[2] Alhuzali, A., Alloqmani, A., Aljabri, M., & Alharbi, F. (2025). In-depth analysis of phishing email detection: Evaluating the performance of machine learning and deep learning models across multiple datasets. [Verify]

[3] (2025). Comparative Investigation of Traditional Machine Learning Models and Transformer Models for Phishing Email Detection. [Preprint]

[4] (2026). Designing effective phishing awareness training: The role of feedback and engagement strategies. International Journal of Human-Computer Studies. [Q1]

[5] Alluqmani, K., et al. (2025). Assessing the Efficacy of Security Awareness Training in Mitigating Phishing Attacks: A Review. International Journal of Advanced Trends in Computer Science and Engineering, 14(3), 177-184. [Verify]

[6] Opara, C., Modesti, P., & Golightly, L. (2025). Evaluating spam filters and stylometric detection of AI-generated phishing emails. Expert Systems with Applications, 276, 127044. [Q1]

[7] (2025). Machine Learning and Watermarking for Accurate Detection of AI-Generated Phishing Emails. Electronics, 14(13), 2611. [Q2]

[8] Uddin, M. A., et al. (2025). Explainable Machine Learning for Phishing Site Detection: A High-Efficiency Approach Using Boosting Models and SHAP. The Journal of Engineering. [Q2]

[9] (2026). Phishing Email Detection Using BERT and RoBERTa. [Q2]

[10] Champa, A. I., Rabbi, M. F., & Zibran, M. F. (2025). Improving phishing email detection performance through deep learning with adaptive optimization. Scientific Reports, 15, 36724. [Q1]

[11] (2026). Hybridizing Explainable AI (XAI) for Intelligent Feature Extraction in Phishing Website Detection. Electronics. [Q2]

[12] (2026). A Temporally Dynamic Feature-Extraction Framework for Phishing Detection with LIME and SHAP Explanations. Future Internet. [Q2]

[13] Alotaibi, S. R., et al. (2025). Explainable artificial intelligence in web phishing classification on secure IoT with cloud-based cyber-physical systems. Alexandria Engineering Journal, 110, 490-505. [Q1]

[14] Sarker, I. H., Janicke, H., Mohsin, A., Gill, A., & Maglaras, L. (2024). Explainable AI for cybersecurity automation, intelligence and trustworthiness in digital twin. ICT Express, 10(4), 935-958. [Q1]

[15] Uddin, M. A., Islam, M. N., Maglaras, L., Janicke, H., & Sarker, I. H. (2025). ExplainableDetector: Exploring transformer-based language modeling approach for SMS spam detection with explainability analysis. Digital Communications and Networks, 11(5), 1504-1518. [Q1]

[16] (2026). An explainable transformer-based model for phishing email detection: A large language model approach. [Q1]

[17] MLPhishChain: A machine learning-based blockchain framework for reducing phishing threats. Frontiers in Blockchain. [Q2]

Appendices

- Appendix A - complete results data is presented in-line in the report body (see Tables 11, 16 and 20) rather than duplicated separately
- Appendix B - Technical and Programmer Manual (separate document)
- Appendix C - User Guide (separate document)
- Appendix D - Consolidated Project Log, Weeks 1-7 (separate document)
- Appendix E - Meeting Minutes #1 to #5
- Appendix F - Evidence screenshots (docs/evidence/)
- Appendix G - Source code repository
- Appendix H – Combined Reference IPO Real Data